

SQLChange: An LLM-Based Assistant for Analyzing SQL Query Modifications

1. Problem Statement and Background Introduction

Modern software systems rely heavily on SQL queries for data retrieval, analytics, and enforcing business logic. As systems evolve, developers frequently modify queries to accommodate new requirements or optimize performance. However, even small changes, such as altering and reordering filters, joins, or aggregations, can have unintended consequences, including incorrect results, degraded performance, or even system failures. Understanding the impact of such modifications is non-trivial and requires extensive familiarity with database systems and awareness of the underlying application context.

SQLChange is a tool that helps developers understand the potential risks, errors and the performance impact of modification of existing SQL queries with structured reasoning and execution-based validation. Given an original and modified SQL query, SQLChange will analyze the dataset schema to provide proper semantic differences between queries, predict performance impacts and flag the potential risks. With SQLChange, a developer can understand the complete implications of their actions before deploying the changes into production.

2. Motivation

These challenges highlight the need for tools that can assist developers in reasoning about SQL query changes. While recent advances in large language models (LLMs) have demonstrated strong capabilities in understanding and generating structured code, they are not inherently reliable at detecting subtle semantic differences or predicting performance implications without additional guidance. This motivates the development of systems that combine LLM-based reasoning with structured analysis to provide more accurate and interpretable insights into how modifications to SQL queries affect behavior.

3. Related Work

There has been extensive research exploring the use of LLMs for building SQL queries and modeling the dataset schema. Here we intend to detect and build upon the gaps of previous research while building SQLChange.

3.1 LLM-Based Query Rewriting and Performance Analysis

Dharwada et al. (2025) use LLMs, such as GPT-4o, for rewriting existing slow SQL queries [1]. The article also utilizes strategies such as the Monte Carlo tree search and database-sensitive prompts to create embeddings for the GPT-4o LLM model. Despite having significant speedups over existing models, LITHE was still quite slow and focused more on rewriting existing queries. On the contrary, in SQLChange we would analyze the risk and performance associated with two different queries entered by the user. As a reach goal, we plan to include LITHE's idea for suggesting improvements/fixes on original queries for performance improvements.

3.2 LLM Understanding of SQL Semantics

Evaluating SQL Understanding in Large Language Models, by Rahaman et al. (2024) evaluates LLMs such as GPT-3.5 and GPT-4 to find syntax error detection and performance prediction in SQL queries and semantics [2]. Their results show that LLM's often struggle with deeper semantic understanding and complex SQL queries. Furthermore, they suggest that LLM's provide good SQL semantic understanding only on the surface level. In SQLChange we want to include structured reasoning with optional execution based evaluation while analyzing the dataset schema rather than relying solely on the SQL query semantics. Additionally, incorporating modern LLM's into SQLChange could improve the reasoning quality on SQL semantics and syntax's.

4. Methodology

SQLChange is a Language Model (LLM) developed to assist developers in comparing two SQL queries. It too can use this information to help determine the possible semantic change, level of risk, and performance implications of the new SQL query as it is being deployed. SQLChange will take two SQL queries and an optional database schema, perform lightweight parsing of their SQL, structural diffing, and then compare the two SQL queries based on filters, joins, projections, aggregations, and order. It will use reasoning with structured LLMs to generate a semantic change summary; risk level rating of the modification; estimate of the performance before and after implementing the modifications, and optionally, run both SQL queries in a test environment to generate execution-based evidence supporting the assessment.

4.1 Query Representation and Difference Analysis

We begin by analyzing the structural differences between the original and modified queries. This may include:

- Changes in WHERE clauses (filters added/removed)
- Changes in JOIN conditions
- Modifications to GROUP BY, ORDER BY, or aggregations
- Column selection differences

A lightweight parsing or diffing approach can be used to highlight these structural changes and provide input to the reasoning module.

4.2 LLM-Based Reasoning

We use a large language model to reason about the impact of the detected changes. The model will be prompted in a structured format to produce step-by-step analysis:

- Identify the type of modification
- Explain how the result set is affected
- Predict risks (e.g., incorrect results, missing constraints)
- Estimate performance changes using heuristics

Structured prompting and fine-tuning will help guide the model toward consistent and interpretable outputs.

4.3 Optional Execution-Based Verification

To ground the model's predictions, we optionally incorporate an execution layer using a lightweight database engine (e.g., SQLite).

For a given query pair:

- Execute both queries on a predefined dataset
- Compare:
 - Result sets (differences in rows or schema)
 - Execution success/failure
 - Approximate runtime

This provides **empirical signals** that can:

- Validate model predictions
- Improve explanation quality
- Serve as part of the evaluation framework

4.4 System Output

The final system produces a structured response including:

- Semantic change summary
- Risk assessment (e.g., low/medium/high)
- Performance estimate
- (Optional) execution-based evidence

5. Evaluation Plan

5.1 Dataset

We will build a benchmark of approximately 250-300 SQL query pairs across three key sources.

1. *Synthetic transformations*: We will take queries from public text-to-SQL datasets benchmarks (e.g. Spider and BIRD) and apply controlled modifications to create a synthetic dataset. Potential modifications include adding/removing filters, swapping join types, changing projections, and altering aggregation queries.
2. *Real-world pairs*: These will come from public GitHub commits that change .sql files, representing real developer changes.
3. *Performance-focused pairs*: We will draw these from query optimization examples where we can measure exact performance differences over scale and speed.

As mentioned above we will execute our query pairs. Specifically, we will execute them against a test database (e.g. SQLite) that has sample schemas and scaled data, which will make it easy to measure runtime differences. We will test against data loaded with multiple scales (e.g. 10k, 100k, 1M rows) so we can evaluate the performance of the query as data size grows.

5.2 Annotation

We will label each pair and evaluate across three dimensions:

1. *Semantic relation*: we will determine this automatically by executing both queries and comparing the resulting sets. Sample values are equivalent, subset, superset, divergent.
2. *Risk level*: We will annotate this by hand using a shared rubric (which we will develop as we see query examples). Samples values are low, medium, high.
3. *Performance direction*: This is measured by runtime, averaged over multiple test executions. This also incorporates how the query performs as we scale the size of our dataset.

5.3 Metrics

Given the ground-truth labels from above, we evaluate our model as follows:

1. *Semantic change detection*: how accurately the system identifies what kind of change was made and how the resulting set is affected.
2. *Risk identification*: how accurately the system classifies risk as low, medium, high. We pay special attention to how often it correctly catches high-risk changes, and weigh the errors based on that.
3. *Performance prediction*: whether the model correctly predicts how the performance will change (e.g. faster, slower) compared to the query runtime, including how predictions hold across different dataset sizes.

5.4 Baselines

To test if these components help, we will compare three versions of the system using the same underlying model:

1. *Zero-shot LLM*: we pass in both queries to the LLM and ask it to compare (face value)
2. *Structured prompt*: we ask the LLM to reason about change type, risk, and performance as separate steps.

3. *Structured prompt + parsed diff*: We build on the above and feed the LLM a summary of structural differences between our query pairs. We are aiming for this to be the core system.

5.5 Error Analysis

We will sample from our failures across each dimension and review them to identify any bias or consistent weaknesses. We can use these to improve our dataset and LLM prompts.

6. Projected Timeline

- **Apr. 28th - May 1st**
 - Finalize Proposal + Begin Setup (Environments, Frameworks, Languages, Models, etc.)
- **May 2nd - May 8th**
 - Core Components (Dataset, Pipeline, Execution Engine, Evaluation Framework)
- **May 9th - May 15th**
 - Integrating components from previous weeks work
- **May 16th - May 22nd**
 - Evaluation + Improvements (Evaluate model performance and make necessary changes and improvements)
- **May 23rd - May 29th**
 - Polish + Reach Goals (i.e. incorporating RAG for better user feedback)
- **May 30th - Jun. 4th**
 - Presentation Preparation

7. Team Roles

- Dev Rathod — dmrathod@ucdavis.edu — Dataset Generation
- Fahim Islam — faislam@ucdavis.edu — Reasoning Pipeline
- Arnav Akula — aakula@ucdavis.edu — Execution Engine
- Ahmed Khan — aekhan@ucdavis.edu — Evaluation Framework
- Devarchith Parashara Batchu — dpbatchu@ucdavis.edu — Integration

7. References

[1] <https://arxiv.org/pdf/2502.12918> (Query Rewriting via LLM's)

[2] <https://arxiv.org/pdf/2410.10680> (Evaluating SQL Understanding in Large Language Models)